



Tema 3 - TCP

Mgtr. Lic. Barrios Walter G.

Confirmaciones, retransmisión y tiempos de espera

- TCP utiliza números de secuencia para identificar la posición de los datos dentro del flujo.
- Los segmentos pueden perderse o llegar fuera de orden.
- El receptor utiliza los números de secuencia para reconstruir correctamente el flujo original.
- TCP confirma únicamente los datos que han sido recibidos de forma continua y sin espacios.
- La confirmación (ACK) indica el número del siguiente octeto que el receptor espera recibir.

Confirmaciones, retransmisión y tiempos de espera

- Confirmaciones (ACK): TCP confirma los datos indicando el número del siguiente octeto que espera recibir.
- Confirmación acumulativa: una confirmación indica que todos los octetos anteriores fueron recibidos correctamente y en orden.
- Retransmisión: si el emisor no recibe una confirmación dentro del tiempo establecido, supone que el segmento se perdió o se dañó y lo vuelve a enviar.
- Timeout: cada segmento enviado tiene un temporizador. Si este expira antes de recibir el ACK correspondiente, se produce la retransmisión.
- Objetivo: estos mecanismos permiten a TCP garantizar una transferencia confiable, incluso cuando los segmentos se pierden, llegan duplicados o fuera de orden.

Confirmaciones, retransmisión y tiempos de espera

- TCP funciona en redes con diferentes características y condiciones.
- Un segmento puede atravesar una sola red local o múltiples redes y routers.
- No es posible conocer de antemano cuánto tardará en llegar una confirmación (ACK).
- El retraso puede variar según el tráfico y la congestión de cada router.
- El tiempo de ida y vuelta (RTT) puede cambiar considerablemente.
- Por esta razón, TCP utiliza un algoritmo de retransmisión adaptativo, ajustando sus tiempos de espera según las condiciones de la red.

Confirmaciones, retransmisión y tiempos de espera

- Para cada conexión, TCP mide el tiempo de ida y vuelta (RTT).
- Registra el momento en que se envía un segmento y cuándo llega su confirmación (ACK).
- La diferencia entre ambos tiempos genera una muestra de RTT.
- TCP utiliza cada nueva muestra para actualizar su estimación del RTT promedio.
- Para realizar esta estimación, utiliza un promedio ponderado que combina:
 - La estimación anterior.
 - La nueva muestra obtenida.

$$RTT = \alpha \times RTT + (1 - \alpha) \times \text{New_Round_Trip_Sample}$$

Confirmaciones, retransmisión y tiempos de espera

- α cercano a 1: da mayor importancia al RTT promedio anterior.
 - El cálculo es más estable.
 - No reacciona rápidamente ante retrasos temporales o aislados.
- α cercano a 0: da mayor importancia a la nueva muestra de RTT.
 - El cálculo es más sensible y rápido ante cambios.
 - Puede reaccionar excesivamente a variaciones momentáneas.
- TCP utiliza la estimación del RTT para calcular el tiempo de espera (timeout).
- El timeout se establece como un valor mayor que el RTT estimado, proporcionando un margen ante posibles variaciones en el retraso.

$$\text{Timeout} = \beta \times \text{RTT}$$

Ambigüedad de ACK y errores en la estimación del RTT

- En teoría, el RTT se obtiene midiendo el tiempo entre el envío de un segmento y la llegada de su ACK.
- El problema aparece cuando TCP realiza una retransmisión.
- El segmento original y su retransmisión contienen los mismos datos y números de secuencia.
- Por eso, cuando llega el ACK, el emisor no puede saber a cuál transmisión corresponde.
- Este problema se denomina ambigüedad de confirmación (ACK ambiguity).
- Asociar el ACK con la transmisión original o con la retransmisión puede producir una medición incorrecta del RTT.

Ambigüedad de ACK y errores en la estimación del RTT

- Asociar el ACK con la transmisión original puede hacer que el RTT estimado aumente continuamente.
- Cada retransmisión puede generar una muestra de RTT demasiado grande, provocando timeouts cada vez mayores.
- Asociar el ACK con la retransmisión más reciente también puede producir errores.
- Si el retraso de la red aumenta repentinamente, el timeout anterior puede ser demasiado corto y provocar una retransmisión innecesaria.

Ambigüedad de ACK y errores en la estimación del RTT

- Si el ACK original llega después de la retransmisión, TCP puede medir un RTT demasiado pequeño.
- Esto hace que el siguiente timeout vuelva a ser demasiado corto, generando nuevas retransmisiones.
- En casos extremos, TCP puede llegar a enviar cada segmento dos veces, aun cuando no exista pérdida real.

Algoritmo de Karn y Timer Backoff

- TCP no actualiza el RTT usando ACKs de segmentos retransmitidos.
- Solo utiliza confirmaciones no ambiguas, es decir, ACKs de segmentos enviados una sola vez.
- Esto evita el problema de determinar si un ACK corresponde al envío original o a una retransmisión.
- Sin embargo, ignorar completamente los RTT de retransmisiones puede provocar que TCP nunca actualice su estimación cuando aumenta el retraso de la red.
- Para solucionarlo, el Algoritmo de Karn utiliza retroceso del temporizador (timer backoff).

Algoritmo de Karn y Timer Backoff

- Si un timeout provoca una retransmisión, TCP aumenta el valor del timeout.
- Cada nueva retransmisión puede volver a incrementar el timeout.
- Normalmente se utiliza un factor multiplicativo γ para aumentar el timeout.
- Las implementaciones suelen establecer un límite máximo para evitar que el timeout crezca excesivamente.

El Algoritmo de Karn evita las mediciones ambiguas de RTT y, mediante timer backoff, aumenta el timeout cuando se producen retransmisiones.

Funcionamiento algoritmo de Karn

- TCP ignora las muestras de RTT de segmentos retransmitidos.
- Utiliza el RTT estimado para calcular un timeout inicial.
- Si ocurre una retransmisión, aplica timer backoff, aumentando el timeout.
- TCP conserva el timeout aumentado para los segmentos siguientes.
- El valor de timeout no se vuelve a calcular con el RTT hasta obtener una muestra válida.

Funcionamiento algoritmo de Karn

- Una muestra válida se obtiene cuando un segmento es confirmado sin haber sido retransmitido.
- En ese momento, TCP actualiza el RTT y restablece el timeout en función de la nueva estimación.
- El algoritmo funciona eficazmente incluso en redes con alta pérdida de paquetes.

Respuesta de TCP ante la congestión

- TCP debe reaccionar no solo a los retrasos entre los extremos, sino también a la congestión de la red.
- La congestión ocurre cuando uno o más routers reciben más datagramas de los que pueden procesar.
- Los datagramas comienzan a acumularse en las colas de los routers, aumentando el retraso.
- Cada router tiene una capacidad de almacenamiento limitada para mantener datagramas en espera.

Respuesta de TCP ante la congestión

- Los datagramas de diferentes conexiones compiten por el mismo espacio de almacenamiento.
- No existe una reserva de recursos exclusiva para cada conexión TCP.
- Si la congestión continúa, las colas se llenan y el router comienza a descartar datagramas.
- Las pérdidas provocadas por congestión pueden generar más retransmisiones y empeorar aún más la situación.

Respuesta de TCP ante la congestión

- Los puntos finales no conocen dónde ni por qué ocurre la congestión; solo perciben un aumento del retraso.
- Un mayor retraso puede provocar timeouts y retransmisiones.
- Las retransmisiones agregan más tráfico y pueden empeorar la congestión.
- Esto puede generar un ciclo:

más tráfico → más retraso → más retransmisiones → más tráfico.

- Si el problema continúa, puede producirse un colapso por congestión, donde la red se vuelve prácticamente inutilizable.

Respuesta de TCP ante la congestión

- TCP evita este problema reduciendo automáticamente la tasa de transmisión cuando detecta congestión.
- Los algoritmos deben considerar que el RTT puede variar incluso cuando la red funciona normalmente.
- TCP utiliza principalmente dos técnicas:
 - Inicio lento (Slow Start)
 - Disminución multiplicativa (Multiplicative Decrease)

Respuesta de TCP ante la congestión

- TCP evita este problema reduciendo automáticamente la tasa de transmisión cuando detecta congestión.
- Los algoritmos deben considerar que el RTT puede variar incluso cuando la red funciona normalmente.
- TCP utiliza principalmente dos técnicas:
 - Inicio lento (Slow Start)
 - Disminución multiplicativa (Multiplicative Decrease)

Ventana de congestión y disminución multiplicativa

- TCP mantiene dos ventanas:
 - Ventana del receptor: indica cuánto buffer puede aceptar el destino.
 - Ventana de congestión: limita el tráfico para evitar sobrecargar la red.
- La ventana permitida es el mínimo entre ambas.
- En condiciones normales, la ventana de congestión puede ser igual a la del receptor.
- Cuando se detecta una pérdida, TCP supone que puede existir congestión.
- Aplica disminución multiplicativa: reduce la ventana de congestión a la mitad por cada pérdida, sin bajar de un segmento.

Ventana de congestión y disminución multiplicativa

- Si las pérdidas continúan, la ventana disminuye exponencialmente, pudiendo llegar a un solo segmento.
- Además, TCP aplica retroceso exponencial del temporizador, aumentando el tiempo antes de cada retransmisión.
- El objetivo es reducir rápidamente el tráfico y dar tiempo a los routers para vaciar las colas congestionadas.

Recuperación de TCP: Slow Start

- Cuando termina la congestión, TCP no duplica inmediatamente la ventana, porque podría provocar nuevas oscilaciones y congestión.
- Utiliza inicio lento (Slow Start) para aumentar progresivamente la transmisión.
- La ventana de congestión comienza con 1 segmento.
- Cada ACK recibido aumenta la ventana en 1 segmento.
- Como se reciben varios ACKs, la ventana crece rápidamente: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow \dots$
- Esto permite que TCP aumente el tráfico de forma controlada, evitando saturar nuevamente la red.

Recuperación de TCP: Slow Start

- En condiciones ideales, la ventana puede crecer exponencialmente por cada RTT.
- Para alcanzar una ventana de N segmentos, se necesitan aproximadamente $\log_2(N)$ RTT.

TCP utiliza Slow Start para recuperar gradualmente la velocidad después de una congestión

Evitando congestión y AIMD

- Cuando la ventana de congestión alcanza la mitad del valor que tenía antes de la congestión, TCP entra en Congestion Avoidance.
- En esta fase, TCP reduce la velocidad de crecimiento de la ventana.
- La ventana aumenta en 1 segmento solo cuando todos los segmentos de la ventana fueron confirmados.
- Este mecanismo combina:
 - Incremento aditivo: aumenta lentamente la ventana.
 - Disminución multiplicativa: ante pérdidas, reduce la ventana significativamente.
- El conjunto se conoce como AIMD (Additive Increase, Multiplicative Decrease).
- Junto con Slow Start, la medición de RTT y el timer backoff, AIMD mejora considerablemente el rendimiento de TCP.

Evitando congestión y AIMD

La arquitectura por capas permite que cada protocolo se concentre en una función específica.

TCP funciona de extremo a extremo

- no necesita conocer los detalles de la ruta que siguen los paquetes.
- tampoco recibe información directa sobre el estado de los routers
- TCP debe inferir la congestión indirectamente, principalmente a partir de retrasos y pérdidas de paquetes.

Las decisiones tomadas por los routers pueden afectar a TCP

Política de Tail Drop

Los routers almacenan temporalmente los datagramas en una cola de memoria mientras esperan poder procesarlos y reenviarlos.

- Si los datagramas llegan más rápido de lo que el router puede reenviarlos, la cola crece.
- Si llegan más lentamente, la cola se reduce.
- Como la memoria del router es limitada, la cola tiene un tamaño máximo.
- Cuando la cola está llena, se aplica la política Tail Drop:

Si la cola de paquetes está llena cuando llega un nuevo datagrama, el router descarta ese datagrama.

Tail Drop y sincronización global en TCP

El nombre Tail Drop proviene de que, cuando la cola del router se llena, se comienzan a descartar los nuevos datagramas que llegan, es decir, la “cola” de la secuencia.

Caso de una sola conexión TCP

- Si se pierden paquetes, TCP detecta la pérdida.
- Reduce su ventana de congestión y puede volver a Slow Start.
- El rendimiento disminuye temporalmente hasta que TCP vuelve a aumentar su tasa de transmisión.

Tail Drop y sincronización global en TCP

Caso de múltiples conexiones TCP

Aquí aparece un problema más importante llamado sincronización global:

- Los paquetes de varias conexiones TCP se mezclan en la cola del router.
- La cola se llena.
- Tail Drop descarta paquetes pertenecientes a varias conexiones al mismo tiempo.
- Todas esas conexiones TCP detectan pérdidas simultáneamente.
- Las conexiones reducen su tasa de transmisión al mismo tiempo y pueden entrar en Slow Start simultáneamente.

Funcionamiento de RED (Random Early Detection)

RED evita esperar a que la cola se llene completamente, como ocurre con Tail Drop. Para ello, utiliza dos umbrales:

- T_{min} : umbral mínimo.
- T_{max} : umbral máximo.

Funcionamiento de RED (Random Early Detection)

El funcionamiento se divide en tres situaciones:

1. Cola menor que T_{min}
 - a. El router acepta el nuevo datagrama y lo agrega a la cola.
 - b. No existe congestión significativa.
2. Cola entre T_{min} y T_{max}
 - a. El router comienza a descartar datagramas de forma aleatoria.
 - b. La probabilidad de descarte, p , depende del tamaño actual de la cola.
 - c. Cuanto más cerca esté la cola de T_{max} , mayor será la probabilidad de descarte.
3. Cola mayor que T_{max}
 - a. El router descarta los nuevos datagramas.
 - b. Esto indica un nivel elevado de congestión.

Conexión TCP: Three-Way Handshake

TCP establece una conexión mediante un proceso de tres mensajes:

1. SYN

- El equipo que inicia la conexión envía un segmento con el bit SYN activado.

2. SYN + ACK

- El otro equipo responde confirmando el SYN recibido (ACK) y enviando su propio SYN.

3. ACK

- El equipo inicial confirma el segundo mensaje.
- En este punto, la conexión queda establecida.

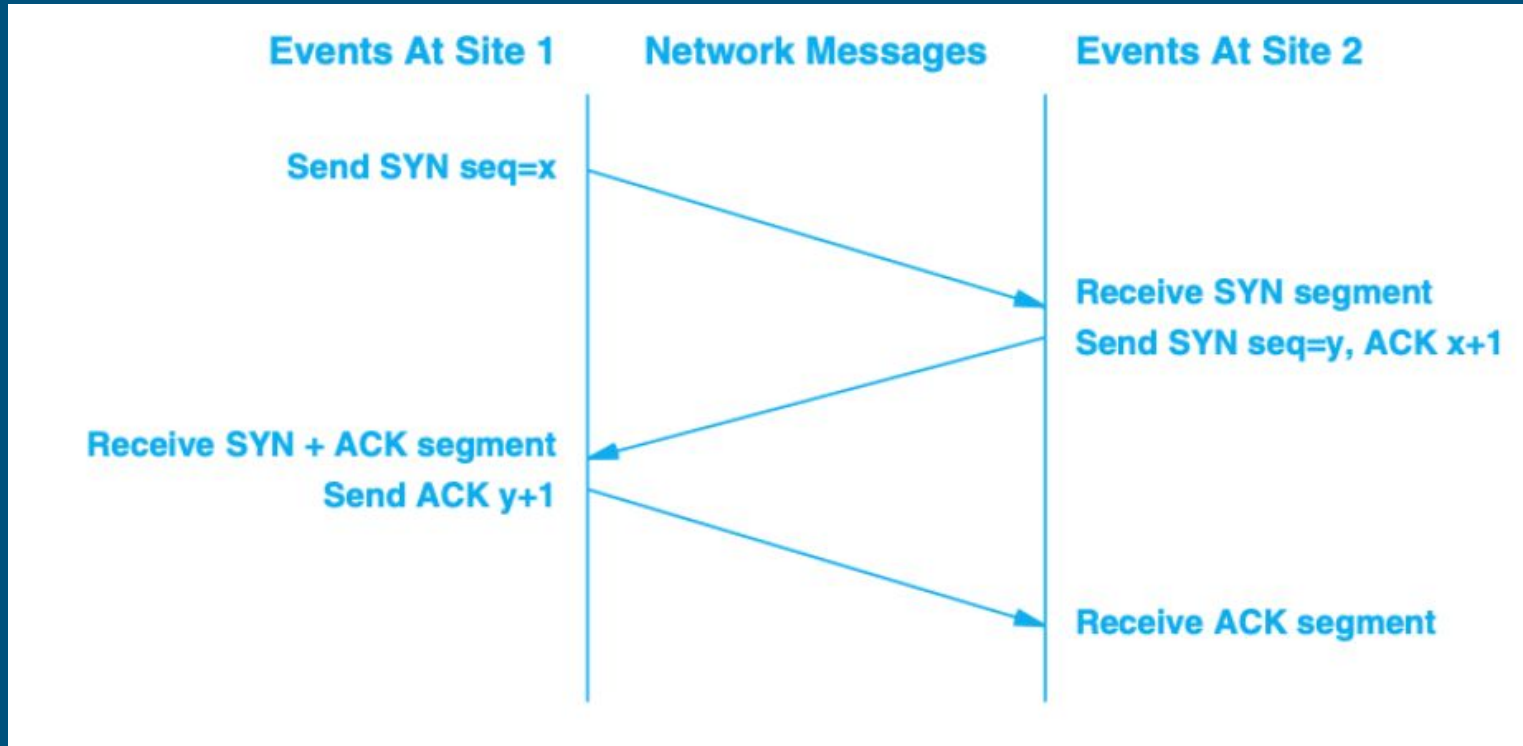
Conexión TCP: Three-Way Handshake

Cliente → SYN → Servidor

Cliente ← SYN + ACK ← Servidor

Cliente → ACK → Servidor

Conexión TCP: Three-Way Handshake



Números de secuencia iniciales en TCP

El three-way handshake cumple dos funciones principales:

1. Garantizar que ambos extremos estén listos para comunicarse.
2. Establecer los números de secuencia iniciales (ISN) para ambos flujos de datos.

Números de secuencia iniciales en TCP

Secuencia del proceso

1. $A \rightarrow B$: SYN
 - a. A elige un número de secuencia inicial x .
 - b. Envía el valor x en el campo de número de secuencia.
2. $B \rightarrow A$: SYN + ACK
 - a. B registra el valor x .
 - b. Elige su propio número inicial y .
 - c. Envía $SEQ = y$.
 - d. Confirma el SYN de A indicando $ACK = x + 1$.
3. $A \rightarrow B$: ACK
 - a. A confirma el SYN de B indicando $ACK = y + 1$.

Números de secuencia iniciales en TCP

¿Por qué los números iniciales son aleatorios?

TCP no puede comenzar siempre con el mismo número de secuencia porque podrían existir segmentos antiguos o duplicados de conexiones anteriores. Elegir números de secuencia iniciales diferentes ayuda a evitar que dichos segmentos sean confundidos con datos de una nueva conexión.

Cierre de una conexión TCP

Proceso básico de cierre

1. Un extremo termina de enviar sus datos pendientes.
2. Espera las confirmaciones correspondientes.
3. Envía un segmento con el bit FIN activado.
4. El otro extremo responde con un ACK.
5. La aplicación receptora es informada de que el otro lado terminó de enviar datos, generalmente mediante un fin de archivo (EOF).
6. La comunicación puede continuar en la dirección opuesta hasta que también se cierre.

Reinicio de una conexión TCP

1. Normalmente, una conexión se cierra mediante FIN, realizando un cierre ordenado (graceful close).
2. En situaciones anormales, TCP puede necesitar interrumpir la conexión inmediatamente.
3. Para ello utiliza el segmento con el bit RST (RESET) activado.
4. El extremo que recibe un RST aborta inmediatamente la conexión.
5. TCP informa del reinicio a la aplicación local que utilizaba la conexión.
6. El reinicio termina la transferencia en ambas direcciones.
7. Es una operación inmediata y no reversible.
8. Al finalizar, TCP libera recursos asociados, como buffers.

Puertos TCP reservados

Port	Keyword	Description
0		Reserved
7	echo	Echo
9	discard	Discard
13	daytime	Daytime
19	chargen	Character Generator
20	ftp-data	File Transfer Protocol (data)
21	ftp	File Transfer Protocol
22	ssh	Secure Shell
23	telnet	Terminal connection
25	smtp	Simple Mail Transport Protocol
37	time	Time
53	domain	Domain name server
80	www	World Wide Web
88	kerberos	Kerberos security service
110	pop3	Post Office Protocol vers. 3
123	ntp	Network Time Protocol
161	snmp	Simple Network Management Protocol
179	bgp	Border Gateway Protocol
443	https	Secure HTTP
860	iscsi	iSCSI (SCSI over IP)
993	imaps	Secure IMAP
995	pop3s	Secure POP3
30301	bittorrent	BitTorrent service